

Unit Testing Framework Assignment

Jawaher Ahmed Alzahrani
1238299428
CSE 565: Software Verification and Validation
02/06/2026

Introduction

The given project applies unit-level testing concepts to achieve software verification and validation .The purpose of this project was not only to run unit tests but also to assess how AI contributes to generated testing and the role of human in enhancing the quality of tests.

Development of an algorithm:

The HeapSort algorithm was written in the Python programming language in the **heapsort.py** file .It works by building a max-heap and exchanging the largest element with the last element. Then, the heap is re-heapified except for the last element .This procedure continues on the remainder of the heap and returns ascending ordered data .

```

heapsort.py ×
heapsort.py > ...
1 |
2 | #This function arranges the input array
3 | # from smallest to largest using the Heapsort algorithm
4 | def heapsort(arr):
5 |     myarr= list(arr) #create copy of th input array
6 |     def heapify(n, i):
7 |         largest = i
8 |         left = 2*i + 1
9 |         right= 2*i + 2
10 |         if left < n and myarr[left] > myarr[largest]:
11 |             largest = left
12 |         if right < n and myarr[right] > myarr[largest]:
13 |             largest = right
14 |         if largest != i:
15 |             myarr[i], myarr[largest] = myarr[largest], myarr[i]
16 |             heapify(n, largest)
17 |     n = len(myarr)
18 |     i = n // 2 - 1
19 |     while i >= 0:
20 |         heapify(n, i)
21 |         i -= 1
22 |     # Extract the large elements one by one using while
23 |     i = n - 1
24 |     while i > 0:
25 |         myarr[0], myarr[i] = myarr[i], myarr[0]
26 |         heapify(i, 0)
27 |         i -= 1
28 |     return myarr #return array after sorting
29 |
30 |

```

Explanation of the unit testing framework and prompt generation:

Pytest was used as the unit testing framework for the Python HeapSort algorithm implementation.

ChatGPT was used as the generative AI tool to create preliminary unit-level test cases and improve them through prompt interactions.

ChatGPT 5.2 Instant ▾

Generate a set of initial unit test cases for a Python implementation of the heapsort algorithm using pytest.

ChatGPT 5.2 Instant ▾

Generate a useful collection of unit test cases for a Python implementation of the heapsort algorithm using pytest. Include duplicates, negative numbers, floating numbers, reverse-sorted input, and random tests.

ChatGPT 5.2 Instant ▾

مشاركة ...

Improve the current pytest tests for the heapsort algorithm by adding more edge cases, stronger property-based assertions, and additional random test scenarios

Explanation of the test cases by AI tool:

ChatGPT was utilized as a generative AI tool to create unit-level test cases for HeapSort. Prompts were given to generate the test cases. These test cases help ensure that our program works as intended for different inputs. Some examples of these inputs are empty lists and single-element lists to check edge cases; lists with duplicate values, negative values, and floats to check correct handling of different numerical types; reverse-order lists; and randomly generated lists to check correct functionality with difficult inputs. It also generated one property-based style test that checks if the output is sorted and has the same elements as the original.

```
python
import pytest
from heapsort import heap_sort

def test_empty_list():
    data = []
    heap_sort(data)
    assert data == []

def test_single_element():
    data = [5]
    heap_sort(data)
    assert data == [5]

def test_already_sorted():
    data = [1, 2, 3, 4, 5]
    heap_sort(data)
    assert data == [1, 2, 3, 4, 5]

def test_reverse_sorted():
    data = [5, 4, 3, 2, 1]
    heap_sort(data)
```

```
ChatGPT 5.2 Instant
heap_sort(data)
assert data == sorted([3, 1, 4, 1, 5, 9, 2])

def test_duplicates():
    data = [4, 2, 2, 8, 4, 2]
    heap_sort(data)
    assert data == [2, 2, 2, 4, 4, 8]

def test_negative_numbers():
    data = [-3, -1, -7, 0, 2]
    heap_sort(data)
    assert data == [-7, -3, -1, 0, 2]

def test_mixed_positive_and_negative():
    data = [10, -2, 3, -5, 0]
    heap_sort(data)
    assert data == [-5, -2, 0, 3, 10]
```

```
ChatGPT 5.2 Instant
python
import random
from collections import Counter
import pytest
from heapsort import heapsort

# ----- Basic tests -----
def test_empty_list():
    assert heapsort([]) == []

def test_single_element():
    assert heapsort([5]) == [5]

# ----- Order-related tests -----
def test_reverse_sorted_input():
    data = [9, 7, 5, 3, 2, -1]
    assert heapsort(data) == sorted(data)

def test_already_sorted_input():
    data = [1, 2, 3, 4, 5, 6]
    assert heapsort(data) == sorted(data)
```

```
ChatGPT 5.2 Instant
@pytest
def test_heapsort(data):
    assert sorted(data) == sorted(data)

# ----- Randomized tests -----
def test_random_lists_repeatable():
    random.seed(42)
    for _ in range(5):
        data = [random.randint(-100, 100) for _ in range(20)]
        assert heapsort(data) == sorted(data)

# ----- Property-based style test -----
def test_sorted_and_permutation_properties():
    data = [random.randint(-50, 50) for _ in range(100)]
    result = heapsort(data)

    # Check sorted order
    assert result == sorted(data)

    # Check permutation (same elements)
    assert Counter(result) == Counter(data)
```

Report out of test case execution:

AI-generated unit test cases from the `test_sort.py` and `test_heapsort.py` were executed using the pytest framework on the Visual Studio Code IDE as the selected integrated development environment. The given screenshots illustrate that all cases passed successfully without any failures.

```
platform darwin -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/jawahar/Desktop/CSE565
collected 12 items

test_heapsort.py .....

===== 12 passed in 0.42s =====
(.venv-2) jawahar@MacBook-Air-Jawahar CSE565 %
```

```
platform darwin -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/jawahar/Desktop/CSE565
collected 8 items

test_sort.py .....

===== 8 passed in 0.42s =====
(.venv-2) jawahar@MacBook-Air-Jawahar CSE565 %
```

```
platform darwin -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/jawahar/Desktop/CSE565
collected 20 items

test_heapsort.py .....
test_sort.py .....

===== 20 passed in 0.83s =====
(.venv-2) jawahar@MacBook-Air-Jawahar CSE565 %
```

```
platform darwin -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/jawahar/Desktop/CSE565
collected 33 items

test_heapsort.py .....
test_improved.py .....
test_sort.py .....

===== 33 passed in 0.83s =====
(.venv-2) jawahar@MacBook-Air-Jawahar CSE565 %
```

Assessment and further improvement of test cases:

When running the preliminary test cases generated by the generative AI tool, it was observed that the tests were adequate to verify the basic functionality of the Heapsort algorithm. Test coverage includes empty lists, single inputs, duplicate inputs, and sorted arrays, as well as reverse-sorted arrays. Despite this, there was limited coverage for unusual and mixed input cases. The improved test cases increased coverage by floating point values, numeric inputs, randomized test cases, and property-based checks to ensure that the output is correctly sorted and that all original elements are preserved after sorting. The full test suite was run with both original test cases and improved test cases with pytest. After test execution, the coverage tool was used to measure code coverage on all test files. The heap sort implementation achieved 100% statement coverage as shown below. Below are screenshots showing the execution results of the improved test cases.

The screenshot shows the VS Code interface with the `test_improved.py` file open. The code includes tests for floats and mixed numbers, randomized tests, and property-based checks. The terminal output shows the execution of the test suite, resulting in 33 passed tests in 0.02s. The coverage report shows 100% coverage for both `heap_sort.py` and `test_improved.py`.

```
def test_floats_and_mixed_numbers(data):
    assert heap_sort(data) == sorted(data)

# ----- Randomized tests -----
def test_random_lists_repeatable():
    random.seed(42)
    for _ in range(10):
        data = [random.randint(-100, 100) for _ in range(20)]
        assert heap_sort(data) == sorted(data)

# ----- Property-based checks -----
def test_sorted_and_permutation_properties():
    data = [random.randint(-50, 50) for _ in range(30)]
    result = heap_sort(data)
```

```
● (venv-2) jawaher@MacBook-Air:~$ coverage erase
coverage run -m pytest -q
coverage report -m
.....
33 passed in 0.02s

Name                               Stmts   Miss  Cover    Missing
-----
heap_sort.py                        24         0   100%
test_improved.py                    33         0   100%
TOTAL                               57         0   100%
```

The screenshot shows the VS Code interface with the `test_heap_sort.py` file open. The code includes tests for random, reverse sorted, and single element inputs. The terminal output shows the execution of the test suite, resulting in 33 passed tests in 0.03s. The coverage report shows 100% coverage for both `heap_sort.py` and `test_heap_sort.py`.

```
1 import random
2 from collections import Counter
3 import pytest
4 from heap_sort import heap_sort

# ----- Basic tests -----
def test_empty_list():
    assert heap_sort([]) == []

def test_single_element():
    assert heap_sort([5]) == [5]
```

```
● (venv-2) jawaher@MacBook-Air:~$ coverage erase
coverage run -m pytest -q
coverage report -m
.....
33 passed in 0.03s

Name                               Stmts   Miss  Cover    Missing
-----
heap_sort.py                        24         0   100%
test_heap_sort.py                   33         0   100%
TOTAL                               57         0   100%
```

An assessment of the generative AI tool

The generative AI tool (ChatGPT) was implemented in this project to support generation of unit-level test cases for the HeapSort algorithm testing through the pytest framework. The tool generated accurate and syntactically correct test cases that covered basic edge cases including empty inputs, single-element lists, duplicate values, negative numbers and sorted arrays. These test cases confirmed the correctness of the algorithm implementation. Although helpful, the AI tool produced very few tests independently at first. Through the use of improved prompts we managed to produce more sophisticated testing approaches including randomized and property-based tests that led to better test coverage. In conclusion, the generative AI tool was helpful as a starting point to generate unit tests through smart prompting and human oversight.